Desarrollo de aplicaciones Android con Kotlin

Information Technology I

Carlos Rojas-Sánchez

Universidad del Mar

Introducción

¿Por qué aprender Kotlin?

- Kotlin es un lenguaje moderno utilizado ampliamente para desarrollar aplicaciones Android.
- Permite escribir código conciso y legible.
- Incluye mecanismos del lenguaje para reducir errores frecuentes, como los relacionados con valores nulos.
- Se integra con Java y con bibliotecas del ecosistema JVM.

Idea central

No comenzamos desde cero: aprovecharemos lo que ya sabemos de Android y aprenderemos a expresarlo con Kotlin.

¿Qué cambia y qué permanece igual?

Permanece

- Android Studio
- Proyecto y recursos
- `Activity` e `Intent`
- XML, vistas y eventos
- Gradle y dependencias

Aprenderemos

- Sintaxis de Kotlin
- Declaración de variables
- Funciones y clases
- Seguridad frente a nulos
- Lambdas y corrutinas

Enfoque del curso

Separaremos los conceptos propios del lenguaje de los conceptos propios de Android.

Java y Kotlin: una comparación inicial

Java

```
int contador = 0;
public void incrementar() {
  contador++;
}
```

Kotlin

```
var contador = 0
fun incrementar() {
  contador++
}
```

- `var` declara una variable reasignable.
- `fun` introduce una función.
- Kotlin puede inferir el tipo a partir del valor inicial.

Ventajas de Kotlin en Android

1. **Menos código repetitivo:** muchas declaraciones son más breves.
2. **Null safety:** el sistema de tipos distingue referencias anulables y no anulables.
3. **Interoperabilidad:** Kotlin puede colaborar con código Java en un mismo proyecto.
4. **Funciones modernas:** lambdas, extensiones y funciones de orden superior.
5. **Asincronía:** las corrutinas facilitan escribir operaciones asíncronas estructuradas.

Kotlin no elimina todos los errores automáticamente: la calidad depende también del diseño, las pruebas y las buenas prácticas.

Objetivos de aprendizaje

Al terminar esta introducción, el estudiante podrá:

- Reconocer la estructura básica de un archivo Kotlin.
- Convertir fragmentos sencillos de Java a Kotlin.
- Declarar variables, utilizar expresiones y crear funciones.
- Aplicar condiciones, ciclos y clases básicas.
- Integrar Kotlin en una aplicación Android existente.

Pregunta de apertura
Si una aplicación Android ya funciona en Java, ¿qué partes deben cambiar necesariamente para comenzar a utilizar Kotlin?

Conceptos básicos de Kotlin

1. Estructura mínima de un programa

Un archivo Kotlin utiliza la extensión `.kt`. Las funciones pueden declararse fuera de una clase.

```
fun main() {
  println("Hola, Kotlin")
}
```

- `fun`: declara una función.
- `main`: punto de entrada de un programa Kotlin independiente.
- `println`: imprime texto y termina la línea.

En Android

Una aplicación no inicia normalmente en `main()`, sino a través de los componentes Android gestionados por el sistema.

2. Variables: `val` y `var`

```
val universidad = "UMAR"  // No se puede reasignar
var intentos = 0          // Se puede reasignar

intentos = intentos + 1
// universidad = "Otra"   // Error: val no se reasigna
```

- Usa `val` por defecto cuando no necesites reasignar la referencia.
- Usa `var` cuando el valor deba cambiar.
- Kotlin infiere el tipo cuando puede deducirlo.

Comparación con Java

`val` se parece a una variable declarada con `final`, aunque la referencia de solo lectura no vuelve inmutable automáticamente al objeto.

3. Tipos de datos básicos

Tipo	Ejemplo
`Int`	`val edad = 20`
`Long`	`val poblacion = 8000000L`
`Double`	`val temperatura = 26.5`
`Float`	`val humedad = 70.0f`
`Boolean`	`val activo = true`
`Char`	`val inicial = 'C'`
`String`	`val nombre = "Ana"`

- Las cadenas usan comillas dobles; los caracteres, comillas simples.
- Los sufijos `L` y `f` indican `Long` y `Float`.
- Kotlin no realiza automáticamente todas las conversiones numéricas entre tipos.

4. Operadores y conversiones

```
val a = 10
val b = 3

val suma = a + b       // 13
val division = a / b   // 3: división entera
val residuo = a % b    // 1
val promedio = a / 2.0 // 5.0

val texto = a.toString()
val numero = "42".toInt()
```

- Operadores aritméticos: `+`, `-`, `*`, `/`, `%`.
- Comparaciones: `==`, `!=`, `<`, `>=`.
- Conversión explícita: `toInt()`, `toDouble()`, `toString()`.

`"abc".toInt()` lanza una excepción; para una conversión segura puede usarse `toIntOrNull()`.

5. Cadenas e interpolación

```
val nombre = "María"
val edad = 21

val mensaje = "Hola, $nombre"
val resumen = "$nombre tiene $edad años"
val calculo = "El próximo año tendrá ${edad + 1}"

println(mensaje)
println(resumen)
```

- `$variable` inserta el valor de una variable.
- `${expresion}` evalúa una expresión dentro de la cadena.
- Las cadenas multilínea pueden escribirse entre comillas triples.

Actividad breve
Construye una cadena que muestre el nombre de una aplicación y su versión.

6. Condiciones: `if` y `when`

```
val calificacion = 8.5
val resultado = if (calificacion >= 6.0) {
  "Aprobado"
} else {
  "No aprobado"
}
val dia = 2
val nombreDia = when (dia) {
  1 -> "Lunes"
  2 -> "Martes"
  3 -> "Miércoles"
  else -> "Otro día"
}
```

- `if` puede utilizarse como expresión y devolver un valor.
- `when` permite elegir entre varias alternativas.
- No se necesita un operador ternario como el de Java.

7. Ciclos y rangos

```
for (i in 1..5) {
  println(i)       // Del 1 al 5, inclusive
}

for (i in 0 until 5) {
  println(i)       // Del 0 al 4
}

var contador = 3
while (contador > 0) {
  println(contador)
  contador--
}
```

- `1..5` incluye ambos extremos.
- `0 until 5` excluye el límite superior.
- `downTo` permite recorrer en orden descendente.

8. Funciones y parámetros

```
fun sumar(a: Int, b: Int): Int {
  return a + b
}

fun saludar(nombre: String): String =
"Hola, $nombre"

fun mostrarMensaje(mensaje: String) {
  println(mensaje)
}
```

- Los parámetros llevan el nombre y después el tipo.
- El tipo de retorno se escribe después de `:`.
- Si no se declara un tipo de retorno, normalmente la función devuelve `Unit`.
- Las funciones de una sola expresión pueden escribirse de forma compacta.

9. Null safety: valores que pueden ser nulos

```
var nombre: String = "Luis"
// nombre = null  // Error de compilación

var apellido: String? = null
val longitud = apellido?.length
val visible = apellido ?: "Sin apellido"

if (apellido != null) {
  println(apellido.length)
}
```

- `String` no admite `null`; `String?` sí.
- `?.` accede a una propiedad solo si el valor no es nulo.
- `?:` proporciona un valor alternativo.
- `!!` fuerza una referencia no nula y puede provocar una excepción.

10. Clases y objetos

```
class Alumno(
val nombre: String,
var calificacion: Double
) {
  fun aprobado(): Boolean {
    return calificacion >= 6.0
  }
}

val alumno = Alumno("Ana", 9.0)
println(alumno.nombre)
println(alumno.aprobado())
```

- El constructor primario se declara junto al nombre de la clase.
- `val` y `var` pueden definir propiedades del objeto.
- Se crea una instancia sin escribir `new`.
- Las clases pueden incluir propiedades, funciones y constructores secundarios.

Resumen y siguiente sección

En esta primera parte revisamos:

- Por qué Kotlin es relevante para Android.
- Las diferencias iniciales entre Java y Kotlin.
- Variables, tipos, operadores y cadenas.
- Condiciones, ciclos, funciones, nulabilidad y clases.